SOFTWARE REQUIREMENTS SPECIFICATION

Automated Asynchronous AI Interviewer

Prepared in accordance with IEEE Std 830-1998 Recommended Practice for Software Requirements Specifications

---

Document Version	1.0 — Draft for Instructor / Sponsor Review
Project Name	Automated Asynchronous AI Interviewer (AAAI)
Development Model	3-Month Vertical-Slice MVP, Agile Sprint Cadence
Team Size	4-Person Student Engineering Team
Prepared By	Project Lead, on behalf of the Engineering Team (Frontend: Soeng Senghorng · Backend: Lim Hokan · AI/Infrastructure: Uy Sovannareach)
Classification	University Capstone / Course Project Deliverable

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Automated Asynchronous AI Interviewer (AAAI), a web application that allows a candidate to complete a structured, AI-graded interview asynchronously — that is, without a live human or synchronous video call — and allows a recruiter to review ranked results afterward.

This document is scoped explicitly around a strict 3-month academic development timeline, executed by a 4-person student engineering team operating at approximately 18 hours per person per week. Every requirement in this specification has been deliberately bound by that constraint: features that could not be reliably designed, built, tested, and demonstrated within the timeline have been formally excluded (see Section 2.5 and the Cut List referenced throughout). The purpose of this SRS is therefore twofold: (a) to serve as the authoritative contract of what the system will and will not do for grading and demonstration purposes, and (b) to give the engineering team a shared, unambiguous reference that allows frontend, backend, and AI/infrastructure workstreams to proceed in parallel from Week 1 onward.

1.2 Document Conventions

This document follows the structure recommended by IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications, adapted for a student capstone context. The following conventions apply throughout:

- Functional requirements are uniquely identified with the prefix SRS-FR-##, in ascending order of introduction, not necessarily order of implementation.
- Non-functional requirements are uniquely identified with the prefix SRS-NFR-##.
- Each requirement is assigned a Priority of Critical, High, Medium, or Low, reflecting its necessity to the MVP vertical slice defined in Section 1.4.
- The keywords "must" and "must not" denote mandatory (Critical/High) requirements; "should" denotes a strong recommendation that may be descoped under schedule pressure without failing the MVP.
- Technology and library names (e.g., FastAPI, PostgreSQL, Whisper, GPT-4o-mini) are used precisely as the team has committed to them; no alternative technology substitutions are considered in-scope for this document.
- Table-formatted requirement blocks are used throughout Sections 4 and 5 to keep user story, inputs, processing, and outputs readable at a glance.

1.3 Intended Audience

This document is intended for the following readers:

- The four members of the student engineering team (Project Lead, Frontend Engineer, Backend Engineer, AI/Infrastructure Engineer), as the primary build reference and sprint-planning input.
- The course instructor, capstone advisor, or academic evaluator, as the artifact demonstrating requirements-engineering rigor for grading purposes.

- Any future contributor onboarding to the project, as a self-contained description of what the system does, why, and under what constraints.
- Prospective pilot recruiters or candidates participating in a demonstration, as a plain-language description of the product's actual scope and data-handling behavior (Sections 2 and 4 in particular).

1.4 Product Scope — The Vertical Slice MVP Model

The AAAI is deliberately built as a Vertical Slice Minimum Viable Product. Rather than building partial layers across many features, the team commits to a single, complete, end-to-end path: a candidate's voice enters the system through a browser microphone, passes through cloud-based transcription and grading, and produces a structured, rankable score visible on a recruiter dashboard. If this thin slice works reliably end-to-end, the project is considered a success at the MVP level, even though many enterprise-grade features remain unbuilt.

This scope decision directly drives Section 4: every functional requirement in this document exists somewhere along that single vertical path — authentication, consent, timed recording, transcription, dynamic follow-up, structured scoring, anti-cheat signal capture, and recruiter review. Explicitly out of scope for this version, by team decision, are: synchronous/live video interviews, webcam or facial-emotion tracking, vision-based processing models, custom or fine-tuned machine learning model training, multi-tenant company accounts, custom PDF export engines, and integration with third-party Enterprise Applicant Tracking Systems (ATS). These exclusions are treated as a formal "Cut List" for this version and any reintroduction would require a new requirements pass, not an in-flight scope change.

1.5 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- React.js — Frontend UI library used for the candidate workspace and recruiter dashboard.
- Tailwind CSS — Utility-first CSS framework used for layout and styling.
- FastAPI (Python) — Backend web framework used for all REST API endpoints.
- PostgreSQL — Relational database system used for Jobs, Candidates, Responses, Scores, and AuditLogs.
- OpenAI Python SDK — Client library used to call OpenAI-hosted models.
- OpenAI Whisper-1 — Cloud speech-to-text transcription model.
- OpenAI GPT-4o-mini — Cloud large language model used for follow-up question generation and structured JSON scoring.
- MDN Web Docs — MediaRecorder API and Page Visibility API (browser-native interfaces referenced in Sections 3.3, 4).

2. Overall Description

2.1 Product Perspective

The AAAI is a new, self-contained web application; it does not replace or extend an existing legacy system. Architecturally, it is a thin orchestration layer between a browser client and a hosted cloud AI provider, deliberately designed around a "Messenger vs. Brain" division of responsibility:

- The application's own code (React frontend, FastAPI backend, PostgreSQL schema) is the "Messenger": it manages user interface state, enforces timing rules, persists data, and moves information between the candidate, the database, and the AI provider.
- The hosted OpenAI models (Whisper-1 and GPT-4o-mini) are the "Brain": all speech-to-text conversion, natural-language reasoning, follow-up question generation, and rubric-based scoring judgment are delegated entirely to the cloud API via the official OpenAI Python SDK.

This division exists because the team's development hardware (student laptops) lacks the GPU capacity to run comparable models locally without severe performance degradation, and because it allows the four-person team to focus their limited hours on product logic rather than machine learning infrastructure. The system is deployed as a single hosted web service (e.g., via Render or Railway) reachable from any modern desktop browser; there is no native mobile application and no on-premises deployment target.

2.2 Product Functions — The Digital Assembly Line

At a high level, the product functions as a linear "digital assembly line" that moves a candidate from invitation to a scored, reviewable record:

- A candidate receives and clicks a passwordless magic-link invitation tied to a specific job (SRS-FR-04).
- The candidate explicitly consents to being recorded and AI-evaluated before any interview content is shown (SRS-FR-01).
- The candidate answers 3–5 pre-seeded Base Questions sequentially under a single 5-minute global timer, recorded directly through the browser microphone (SRS-FR-02, SRS-FR-05).
- Recorded audio is uploaded, validated, and transcribed in the background via Whisper (SRS-FR-06, SRS-FR-07), pacing the candidate with a natural-feeling "Processing..." state (SRS-FR-17).
- GPT-4o-mini generates one contextual, single-turn follow-up question from the candidate's own base answers, answered under a strict 2:30 window (SRS-FR-08, SRS-FR-09).
- GPT-4o-mini produces a structured, schema-validated JSON scorecard across four traits — Technical Skill, Communication, Problem Solving, Job Fit — incorporating a robotic-language penalty where applicable (SRS-FR-03, SRS-FR-10).
- Anti-cheat signals (clipboard blocking, tab-out tracking) are captured continuously and written to an immutable audit trail (SRS-FR-11, SRS-FR-12, SRS-FR-13).
- An authenticated recruiter reviews a ranked leaderboard, drills into individual transcripts and scores, plays back raw audio, and sees manual-review flags where the AI's confidence or the candidate's behavior warrants human judgment (SRS-FR-14, SRS-FR-15).
- Throughout, a budget kill-switch continuously guards against runaway OpenAI spend (SRS-FR-16).

2.3 User Classes and Characteristics

User Class	Characteristics	System Interaction
Candidate	External, generally non-technical user. Owns a modern browser and a microphone. Motivated to present themselves well; may be tempted to seek outside help.	Receives a magic link, consents, answers base and follow-up questions via voice, has zero visibility into scoring internals.
Recruiter	Internal/trusted user with an authenticated account. Time-constrained; needs fast, comparable signals across many candidates.	Authenticates via magic link, views the ranked leaderboard, drills into transcripts/scores, plays raw audio, and overrides AI judgment when flagged.
Project Lead (System Administrator role)	Technical, trusted user responsible for rubric design and AI quality assurance.	Configures Base Questions and rubrics, authors and revises system prompts, runs jailbreak/injection/consistency test batches.

2.4 Operating Environment

- Client: Any evergreen desktop web browser with microphone support — Google Chrome, Mozilla Firefox, and Apple Safari are the explicitly targeted browsers (see SRS-NFR-08).
- Client hardware: A standard laptop or desktop with a functioning microphone; no dedicated GPU or specialized hardware is required on the client.
- Server: A single hosted cloud instance (e.g., Render or Railway) running the Python/FastAPI backend, connected to a managed or co-hosted PostgreSQL instance.
- External dependency: The OpenAI hosted API platform (Whisper-1 and GPT-4o-mini endpoints), reachable over HTTPS; the system has a hard runtime dependency on this third-party service being available.
- No native mobile app, desktop installer, or offline mode is provided in this version.

2.5 Design and Implementation Constraints

- Cloud-only AI: All transcription and reasoning must use the OpenAI-hosted Whisper-1 and GPT-4o-mini models via the official OpenAI Python SDK. No local model hosting, no custom fine-tuning, and no alternative model providers are in scope.
- No video: The system must not implement webcam capture, live video streaming, facial recognition, or emotion-detection/vision processing of any kind, per the approved Cut List.
- Relational integrity: PostgreSQL must be used with explicit Foreign Key constraints linking Jobs, Candidates, Responses, Scores, and AuditLogs; no schema-less or document-store persistence is permitted for this data.
- Fixed timing windows: The three "Interview Physics" windows — 5:00 base round, 15-second processing pause, 2:30 follow-up — are treated as fixed product constants, not configurable settings, for this MVP version.

- Budget ceiling: All AI-calling code must operate under the assumption that a hard $10.00/month spending ceiling exists and may trigger mid-session (SRS-FR-16).
- Team capacity: The design must be implementable by a 4-person team at roughly 18 hours/week each within a 3-month window; any requirement that cannot reasonably be built at that capacity is out of scope for this version.

2.6 Assumptions and Dependencies

- It is assumed that candidates have access to a private space, a working microphone, and a stable internet connection sufficient to upload a 10–15 MB audio file within the timing windows defined in Section 4.
- It is assumed that the OpenAI API platform remains available and within its documented rate limits for the duration of any pilot or demonstration; the system has no fallback AI provider.
- It is assumed that the pre-seeded Base Questions and scoring rubric are authored and finalized by the Project Lead before the first pilot session, as these are treated as configuration input, not runtime user input.
- It is assumed that recruiters using the system are trusted, vetted users; this version does not implement fine-grained recruiter permission tiers beyond a single authenticated recruiter role.
- The project depends on continued access to Render/Railway (or an equivalent) hosting platform for the full 3-month build-and-deploy cycle.

3. External Interface Requirements

3.1 User Interfaces

The system exposes two distinct interface surfaces, both built with React.js and styled with Tailwind CSS for a consistent, lightweight visual language:

Candidate Interview Workspace

- A blocking consent screen presented before any question content (supports SRS-FR-01).
- A single-column question view showing the current question text, a record/stop control, and a persistent, always-visible countdown timer (supports SRS-FR-02, SRS-FR-05).
- A non-interactive "Processing..." transition screen shown between submission and the next question or completion (supports SRS-FR-17).
- A completion/thank-you screen shown once all base and follow-up responses have been submitted.

Recruiter Dashboard

- A magic-link sign-in screen (supports SRS-FR-04).
- A ranked leaderboard table per Job_ID, showing candidate name, aggregate score, cumulative TAB_OUT count, and a manual-review flag indicator where applicable (supports SRS-FR-14, SRS-FR-15).
- A per-candidate detail drawer showing the full transcript, per-trait scores with AI-generated rationale, and an embedded audio player for each response (supports SRS-FR-14, SRS-FR-15).

3.2 Hardware Interfaces

The system has no direct, driver-level hardware interface of its own. Its sole hardware dependency is indirect, through the browser: the candidate's device must expose a functioning microphone that the browser can access via standard OS-level audio input permissions. No specialized recording hardware, headset, or webcam is required or used, consistent with the no-video Cut List decision in Section 2.5.

3.3 Software Interfaces

Interface	Direction / Role	Purpose in This System
Browser MediaRecorder API	Client-side, browser-native	Captures candidate microphone audio into an uploadable Blob (SRS-FR-02).
Browser Page Visibility API	Client-side, browser-native	Detects tab/window blur events for anti-cheat tracking (SRS-FR-12).
FastAPI REST Endpoints	Client ↔ Server	Serves authentication, question retrieval, audio upload, and dashboard data as JSON over HTTPS.
PostgreSQL Driver	Server ↔ Database	Persists Jobs, Candidates, Responses, Scores, and AuditLogs with enforced Foreign Keys.

Interface	Direction / Role	Purpose in This System
OpenAI Python SDK	Server ↔ Cloud AI	Dispatches Whisper-1 transcription calls and GPT-4o-mini follow-up/grading calls.
Email Delivery Service	Server → External	Delivers magic-link authentication emails to candidates and recruiters (SRS-FR-04).

3.4 Communications Interfaces

- All client-server communication uses asynchronous HTTP/HTTPS requests exchanging RESTful JSON payloads; there is no persistent WebSocket or real-time streaming channel, consistent with the explicit exclusion of synchronous/live communication from scope.
- Audio file transfer from client to server uses multipart form upload over HTTPS.
- Server-to-OpenAI communication uses HTTPS calls through the OpenAI Python SDK, authenticated via an API key isolated in server-side environment variables.
- Background job status is communicated to the client via short-interval polling rather than a push/streaming mechanism, matching the 15-second asynchronous processing model (SRS-FR-17).

4. System Features / Functional Requirements

This section enumerates every functional requirement required to realize the vertical-slice MVP described in Section 1.4 and the digital assembly line described in Section 2.2. Each requirement is presented as a self-contained specification block stating its user story, preconditions, inputs, step-by-step processing behavior, outputs, and acceptance criteria, so that it can be picked up independently by any team member as a unit of implementation work.

SRS-FR-01 — Candidate Consent & Data Privacy Acknowledgment *[Priority: Critical]*	
User Story	As a candidate, I must explicitly agree to be recorded and evaluated by an AI system before any interview data is collected, so that my participation is informed and legally documented.
Preconditions	• Candidate has followed a valid, unexpired magic-link invitation for a specific Job_ID.
Inputs	• Candidate's affirmative click/tap on a consent checkbox • Timestamp of the consent action • Candidate_ID and Job_ID from the session context
Processing Steps	1. Render a blocking consent modal describing audio/text recording, AI-based transcription, and AI-based scoring before any question is shown. 2. Disable the 'Continue' control until the checkbox is explicitly checked (no pre-checked defaults). 3. On confirmation, write a consent record (Candidate_ID, Job_ID, timestamp, consent_version) to the database. 4. Redirect the candidate to the Base Question workspace only after the consent record is persisted.
Outputs	• Persisted consent row usable as an audit trail • Unlocked interview workspace
Acceptance Criteria	• A candidate cannot reach Base Question 1 without a persisted consent record. • Attempting to bypass consent via direct URL navigation is rejected server-side (401/403).

SRS-FR-02 — Browser Audio Standardization via MediaRecorder API *[Priority: Critical]*	
User Story	As a candidate, I want my spoken answer captured directly in the browser without installing software, so the interview works on any laptop with a microphone.
Preconditions	• Browser has granted microphone permission. • Candidate is within an active, timed question window.
Inputs	• Raw microphone stream from the browser's MediaDevices.getUserMedia API • Selected MIME type (audio/webm or audio/mp4 fallback for Safari)
Processing Steps	1. Initialize a MediaRecorder instance scoped to the current question at the moment the countdown starts.

SRS-FR-02 — Browser Audio Standardization via MediaRecorder API *[Priority: Critical]*	
	2. Buffer audio chunks client-side using the ondataavailable event. 3. On timer expiry or manual submit, stop the recorder and assemble a single Blob. 4. Standardize the output container/codec where the browser allows, falling back gracefully on Safari's supported formats. 5. Attach Candidate_ID, Job_ID, and Question_ID metadata and hand the Blob to the upload routine (FR-06).
Outputs	• A single audio Blob per question ready for upload • Client-side recording-state indicator (recording / stopped / uploading)
Acceptance Criteria	• Recording starts and stops without requiring a page reload. • Recorded files play back correctly in Chrome, Firefox, and Safari.

SRS-FR-03 — Structured JSON Scorecard Generation via response_format *[Priority: Critical]*	
User Story	As a recruiter, I want every candidate scored on the same four traits in a strictly structured format, so that I can compare candidates on a single dashboard without manually parsing free text.
Preconditions	• All base and follow-up transcripts for a candidate session exist in the database.
Inputs	• Full transcript set for the session • Job-specific scoring rubric/system prompt • Robotic-language heuristic flags (FR-10 output)
Processing Steps	1. Construct a grading prompt combining the transcripts, the four-trait rubric, and the anti-templating instruction. 2. Call the GPT-4o-mini Chat Completion endpoint with response_format set to json_object and temperature fixed at 0.0 for consistency. 3. Validate the returned JSON against an expected schema (four integer fields 1–5, plus a rationale string per trait). 4. On schema validation failure, retry once with a corrective system message; on second failure, flag the session for manual recruiter review instead of guessing. 5. Persist the validated scorecard to the Scores table, foreign-keyed to Candidate_ID and Job_ID.
Outputs	• A structured scorecard: Technical Skill, Communication, Problem Solving, Job Fit — each 1 to 5, with a short rationale • A manual-review flag when structured parsing fails
Acceptance Criteria	• 100% of persisted scorecards conform to the fixed JSON schema. • No free-text score ever reaches the recruiter dashboard unparsed.

SRS-FR-04 — Passwordless Authentication via Email Magic Link *[Priority: Critical]*	
User Story	As a candidate or recruiter, I want to sign in with a single emailed link instead of managing a password, so onboarding is fast and there is no password database to secure.
Preconditions	• A valid email address is on file (candidate invited to a Job_ID, or recruiter provisioned as a system user).
Inputs	• Email address • Requested role context (candidate interview link vs. recruiter dashboard link)
Processing Steps	1. Generate a cryptographically random, single-use token bound to the email, role, and (for candidates) Job_ID. 2. Email a link containing the token with a defined expiry window. 3. On click, validate the token has not expired or been previously consumed. 4. Issue a short-lived session credential (signed cookie or JWT) scoped to the appropriate role. 5. Invalidate the token immediately after first successful use.
Outputs	• An authenticated session for the requesting role • A rejected/expired-link error state for invalid tokens
Acceptance Criteria	• A used or expired token can never authenticate a second session. • Candidate tokens cannot be reused to access the recruiter dashboard and vice versa.

SRS-FR-05 — Sequential Base Question Presentation & Global Countdown *[Priority: Critical]*	
User Story	As a candidate, I want to see 3–5 standard questions one at a time under a single visible timer, so I understand exactly how much time I have left for the entire base round.
Preconditions	• Consent (FR-01) has been recorded. • The Job_ID has 3–5 pre-seeded Base Questions configured.
Inputs	• Ordered list of pre-seeded Base Questions for the Job_ID • Client clock tick events
Processing Steps	1. Fetch the ordered question set and start a single global countdown at 5:00 (300 seconds) the moment Question 1 is displayed. 2. Present each question sequentially; advancing to the next question does not reset or extend the shared countdown. 3. Allow the candidate to submit a response and move forward at any point before the countdown reaches zero. 4. If the countdown reaches zero mid-question, force-submit whatever audio has been captured so far and end the base round. 5. Persist a Response row per question referencing the audio artifact from FR-02.
Outputs	• A visible, continuously updating countdown UI

SRS-FR-05 — Sequential Base Question Presentation & Global Countdown *[Priority: Critical]*	
	• A complete or partial Response record for each Base Question
Acceptance Criteria	• The 5:00 window is enforced globally across all base questions, not per-question. • A hard stop occurs automatically at 0:00 with no way to extend it client-side.

SRS-FR-06 — Asynchronous Audio Upload & File Validation Pipeline *[Priority: High]*	
User Story	As a backend system, I need to accept, validate, and store candidate audio reliably, so oversized or corrupt files never destabilize storage or downstream AI calls.
Preconditions	• A recorded audio Blob exists client-side (FR-02).
Inputs	• Audio Blob (expected 10–15 MB given the 5-minute window) • Candidate_ID, Job_ID, Question_ID metadata
Processing Steps	1. Stream the upload via a multipart FastAPI endpoint rather than buffering the entire file in memory where avoidable. 2. Reject any file exceeding the hard 20 MB ceiling with a 413 response before it touches persistent storage. 3. Validate MIME type against an accepted allow-list (webm, mp4, wav, m4a). 4. Store the file in object storage or a designated media directory, recording only the file path/reference in PostgreSQL. 5. Enqueue the stored file for background transcription (FR-07).
Outputs	• A persisted, path-referenced Responses row • A 413/415 rejection for oversized or invalid files
Acceptance Criteria	• Files above 20 MB are never written to disk. • A successful upload always results in exactly one Responses row per question.

SRS-FR-07 — Speech-to-Text Transcription via Whisper API *[Priority: Critical]*	
User Story	As the grading engine, I need accurate text transcripts of what the candidate said, so scoring is based on content rather than raw audio.
Preconditions	• A validated audio file exists (FR-06).
Inputs	• Stored audio file reference
Processing Steps	1. Submit the audio file asynchronously to the OpenAI Whisper-1 transcription endpoint inside the 15-second processing wrapper (FR-17). 2. Capture the returned transcript text and, where available, a detected-language field. 3. Handle empty, silent, or unintelligible audio by storing a null-content flag rather than fabricating a transcript. 4. Persist the transcript against the corresponding Responses row.

SRS-FR-07 — Speech-to-Text Transcription via Whisper API *[Priority: Critical]*	
Outputs	• Plain-text transcript stored against the Response • A 'no speech detected' flag for empty/failed transcriptions
Acceptance Criteria	• Every non-empty audio submission produces a stored transcript or an explicit failure flag — never a silent data gap.

SRS-FR-08 — Single-Turn Dynamic Follow-Up Question Generation *[Priority: High]*	
User Story	As a recruiter, I want the AI to ask one contextual follow-up drawn from the candidate's own answers, so I can see how they think beyond a scripted response.
Preconditions	• All Base Question transcripts (FR-07) for the session are available.
Inputs	• Concatenated base-round transcripts • Job-specific context/system prompt
Processing Steps	1. Send the base transcripts to GPT-4o-mini with a system prompt instructing exactly one targeted, technically specific follow-up question. 2. Constrain the response to plain text (no JSON needed at this stage) and a strict output length limit. 3. Sanitize the model output against basic prompt-injection artifacts (e.g., stray instruction-style text) before display. 4. Present the single follow-up question to the candidate and immediately start the 2:30 recording window (FR-09).
Outputs	• One rendered follow-up question • A new Response row of type 'follow_up' pending audio
Acceptance Criteria	• Exactly one follow-up question is generated and shown per session — never zero, never multiple.

SRS-FR-09 — Dynamic Follow-Up Recording Window Enforcement *[Priority: High]*	
User Story	As the system, I need to strictly cap the follow-up answer time, so a candidate cannot pause to search for or generate an answer externally.
Preconditions	• The follow-up question (FR-08) has been displayed.
Inputs	• Client clock tick events • Recorded audio Blob for the follow-up
Processing Steps	1. Start a hardcoded 2:30 (150 second) countdown the instant the follow-up question renders. 2. Reuse the MediaRecorder pipeline (FR-02) scoped to this single question. 3. Force-submit captured audio automatically at 0:00 with no extension mechanism. 4. Route the resulting Blob through the same upload and validation pipeline (FR-06).

SRS-FR-09 — Dynamic Follow-Up Recording Window Enforcement *[Priority: High]*	
Outputs	• A time-bounded follow-up Response record
Acceptance Criteria	• No follow-up recording session can exceed 150 seconds under any client condition.

SRS-FR-10 — Robotic / Templated Language Detection Heuristic *[Priority: Medium]*	
User Story	As a recruiter, I want candidates who paste AI-generated or memorized textbook answers to score lower on communication, so the metric reflects authentic speech.
Preconditions	• A full transcript set exists for the session (FR-07).
Inputs	• Transcript text • A defined list of structural marker phrases (e.g., 'Furthermore,' 'In conclusion,' rigid three-part essay structure)
Processing Steps	1. Embed explicit heuristic instructions inside the GPT-4o-mini grading system prompt (FR-03) directing it to detect artificial phrasing loops, unnaturally uniform sentence structure, and textbook transition patterns. 2. Instruct the model to apply a documented down-weighting adjustment to the Communication trait when such patterns dominate the transcript. 3. Record which heuristic triggered inside the rationale field of the scorecard for transparency.
Outputs	• A communication score adjusted for templated phrasing, with the trigger reason recorded
Acceptance Criteria	• A transcript saturated with the defined marker phrases receives a visibly lower Communication score than an equivalent natural-language answer, and the rationale names the trigger.

SRS-FR-11 — Clipboard & Context-Menu Lock (Anti-Cheat) *[Priority: Medium]*	
User Story	As a recruiter, I want to prevent candidates from pasting pre-written answers into any text fallback field, so typed responses reflect the candidate's own real-time composition.
Preconditions	• Candidate is inside an active interview session.
Inputs	• Browser paste events (Ctrl+V / Cmd+V) • Browser contextmenu (right-click) events
Processing Steps	1. Attach global event listeners on the interview workspace that call preventDefault() on paste and contextmenu events. 2. Apply the listeners to every text fallback field and the page body for the duration of the session. 3. Remove the listeners cleanly once the session ends to avoid affecting the recruiter's own browser experience.

SRS-FR-11 — Clipboard & Context-Menu Lock (Anti-Cheat) *[Priority: Medium]*	
Outputs	• A workspace where paste and right-click actions are inert
Acceptance Criteria	• No text can be inserted into a fallback field via paste or a right-click 'Paste' menu item.

SRS-FR-12 — Tab Visibility Tracking (Page Visibility API) *[Priority: Medium]*	
User Story	As a recruiter, I want to know if a candidate switched tabs during the interview, so I can factor potential outside assistance into my evaluation.
Preconditions	• Candidate is inside an active, timed question window.
Inputs	• Browser 'visibilitychange' events • document.hidden state
Processing Steps	1. Register a visibilitychange listener for the duration of the interview session. 2. On each transition to hidden, capture Candidate_ID, Job_ID, Question_ID, and a precise timestamp. 3. Send a lightweight background request appending a 'TAB_OUT' event row to the AuditLogs table without interrupting the active recording. 4. Aggregate the cumulative TAB_OUT count per session for display on the recruiter dashboard (FR-14).
Outputs	• One AuditLogs row per tab-out event • A cumulative tab-out counter visible to recruiters
Acceptance Criteria	• Every tab/window blur event during an active question is captured with no missed or duplicated entries.

SRS-FR-13 — Immutable Audit Log Persistence *[Priority: Critical]*	
User Story	As a recruiter, I need a tamper-proof record of exactly what was sent to and received from the AI, so I can defend or override any score I disagree with.
Preconditions	• Any AI API call (Whisper or GPT-4o-mini) or anti-cheat event has occurred.
Inputs	• Raw request payloads sent to OpenAI • Raw response payloads received from OpenAI • Anti-cheat event data (FR-12)
Processing Steps	1. Write every AI request/response pair, and every anti-cheat event, as a new row in an append-only AuditLogs table. 2. Enforce immutability at the application layer: no UPDATE or DELETE code path is exposed for this table. 3. Foreign-key every row to Candidate_ID and Job_ID so logs can be retrieved per session.

SRS-FR-13 — Immutable Audit Log Persistence *[Priority: Critical]*	
Outputs	• A complete, chronologically ordered, non-editable audit trail per candidate session
Acceptance Criteria	• No application code path can modify or delete an existing AuditLogs row. • Every scored session has a retrievable full log of prompts and payloads.

SRS-FR-14 — Authenticated Recruiter Leaderboard & Scorecard Dashboard *[Priority: High]*	
User Story	As a recruiter, I want a ranked list of candidates per job with drill-down transcripts and scores, so I can quickly shortlist top performers.
Preconditions	• Recruiter has authenticated via FR-04. • At least one candidate session has a persisted scorecard (FR-03).
Inputs	• Job_ID filter selection • Aggregated Scores, Responses, and AuditLogs data
Processing Steps	1. Query and rank candidates per Job_ID by an aggregate of the four trait scores. 2. Surface the cumulative TAB_OUT count and any manual-review flags (FR-15) inline on the leaderboard row. 3. Provide a drill-down panel per candidate showing transcripts, per-trait scores with rationale, and raw audio playback controls. 4. Restrict all dashboard routes to authenticated recruiter sessions only.
Outputs	• A ranked, filterable leaderboard view • A per-candidate detail drawer with transcripts, scores, and audio
Acceptance Criteria	• Leaderboard ranking updates as soon as a new scorecard is persisted. • Unauthenticated requests to any dashboard endpoint are rejected.

SRS-FR-15 — Human-in-the-Loop Manual Review & Audio Playback Override *[Priority: High]*	
User Story	As a recruiter, I want to be alerted when the AI's confidence is low or a candidate's communication score is unusually low, so I can personally listen before making a final call.
Preconditions	• A scorecard (FR-03) has been generated, or a JSON validation failure occurred.
Inputs	• Scorecard trait values • Cumulative TAB_OUT count (FR-12) • JSON-parse failure state (FR-03)
Processing Steps	1. Evaluate each new scorecard against defined flag thresholds (low Communication score, high TAB_OUT count, or a grading-schema failure). 2. Mark the candidate row with a visible 'Needs Review' flag on the leaderboard when a threshold is crossed.

SRS-FR-15 — Human-in-the-Loop Manual Review & Audio Playback Override *[Priority: High]*	
	3. Expose a native audio player against each stored response so the recruiter can play back the original recording directly. 4. Allow the recruiter to view the flag reason(s) alongside the AI-generated rationale.
Outputs	• A 'Needs Review' indicator with a stated reason • Playable raw audio per response
Acceptance Criteria	• Every flagged candidate displays at least one human-readable reason for the flag. • Audio playback works for every stored response without requiring a file download.

SRS-FR-16 — Budget Kill-Switch & Token Usage Monitoring Middleware *[Priority: Critical]*	
User Story	As the project owner, I need automated protection against runaway API costs, so a bug can never generate an unexpected bill.
Preconditions	• An OpenAI API call is about to be made.
Inputs	• Running monthly token/cost estimate • Configured $10.00 monthly ceiling
Processing Steps	1. Increment a running cost-estimate counter in backend middleware after every Whisper or GPT-4o-mini call, using per-call token/duration pricing. 2. Check the running total against the $10.00 ceiling before dispatching each new AI request. 3. When the estimate reaches or exceeds the ceiling, block all further outbound AI calls for the remainder of the billing cycle. 4. Surface a persistent 'AI paused due to budget cap' banner on the recruiter dashboard when frozen. 5. Mirror the same ceiling as a hard spending limit configured directly in the OpenAI platform dashboard as a second, provider-side safeguard.
Outputs	• A blocked-call state once the ceiling is reached • A visible budget-paused notification for recruiters
Acceptance Criteria	• No AI call is dispatched once the internal estimate reaches the ceiling. • The provider-side $10.00 hard cap exists as an independent backstop.

SRS-FR-17 — Asynchronous Processing Timeout Wrapper *[Priority: High]*	
User Story	As a candidate, I want a smooth 'Processing...' pause between my answer and the next question instead of a frozen or error screen, so the interview feels natural even when the AI is briefly slow.
Preconditions	• A response has been submitted and queued for transcription/grading.

SRS-FR-17 — Asynchronous Processing Timeout Wrapper *[Priority: High]*	
Inputs	• Background job state (queued / running / complete)
Processing Steps	1. Wrap the Whisper transcription and GPT follow-up/grading calls in a background async task rather than holding open the client's HTTP connection. 2. Render a client-side 'Processing...' state for up to 15 seconds, matching natural interview pacing. 3. If the AI pipeline has not returned within 15 seconds, keep the job running server-side and continue polling instead of raising a network timeout to the candidate. 4. Push the result to the client (next question, or scoring completion) as soon as the background job resolves.
Outputs	• A non-blocking 'Processing...' UI state • A completed job result delivered without a client-side timeout error
Acceptance Criteria	• No candidate ever sees a raw network-timeout error even when an AI call exceeds 15 seconds. • The client always eventually receives a result or an explicit, user-readable failure message.

5. Non-Functional Requirements

Non-functional requirements describe how the system must behave — its quality attributes — rather than what individual features it must have. For a student-built product handling real audio recordings, real personal data, and a real (if small) financial exposure, these constraints are treated as equally binding as the functional requirements in Section 4.

SRS-NFR-01 — Data Integrity & Immutable Audit Logging *[Priority: Critical]*	
Requirement Statement	All AuditLogs records (AI request/response payloads and anti-cheat events) must be append-only and permanently traceable to a specific Candidate_ID and Job_ID via enforced foreign-key constraints.
Rationale	Because AI grading can misjudge non-native speakers, speech impediments, or unusual phrasing, recruiters must be able to reconstruct exactly what happened in a session to override a decision confidently and fairly.
System Behavior / Implementation	• No application code path exposes UPDATE or DELETE for the AuditLogs table. • Foreign keys on Candidate_ID and Job_ID are enforced at the PostgreSQL schema level, not only in application code. • Raw prompts and raw model payloads are stored verbatim, not summarized.
Verification / Fit Criterion	A database-level attempt to modify or delete an existing AuditLogs row fails; every scored candidate has a fully retrievable, chronologically intact log.

SRS-NFR-02 — Performance & Latency Boundary Handling *[Priority: Critical]*	
Requirement Statement	The system must never expose a raw network timeout to the candidate; any AI processing step exceeding 15 seconds must be handled asynchronously with a graceful in-progress state.
Rationale	Cloud AI response time is variable and outside the team's control. A frozen browser or an HTTP timeout error would look like a broken product during a live demo or a real candidate's interview.
System Behavior / Implementation	• Long-running AI calls run as background tasks, decoupled from the request/response cycle that serves the browser. • The frontend polls or listens for job completion rather than holding a single open connection. • A defined maximum reasonable wait (e.g., 60–90 seconds) triggers a user-facing 'taking longer than usual' message rather than an indefinite spinner.
Verification / Fit Criterion	Zero unhandled network-timeout errors are observed across a test batch of at least 20 full interview sessions.

SRS-NFR-03 — Financial API Billing Ceilings *[Priority: Critical]*	
Requirement Statement	Total monthly OpenAI API spend must not exceed $10.00, enforced by both application-level monitoring and a provider-side hard spending cap.

SRS-NFR-03 — Financial API Billing Ceilings *[Priority: Critical]*	
Rationale	A four-person student team is personally financially exposed; an accidental infinite loop calling a paid API overnight must be structurally impossible to turn into an unexpected bill.
System Behavior / Implementation	• Backend middleware estimates per-call cost from token/duration usage and maintains a running monthly total. • The OpenAI developer dashboard has an independent $10.00 hard cap configured outside of application code. • Normal development/testing activity is expected to stay in the $2–$3/month range, leaving comfortable headroom under the ceiling.
Verification / Fit Criterion	Simulated load testing that would otherwise exceed the ceiling results in calls being frozen before the $10.00 threshold is crossed, with the provider-side cap as a verified independent backstop.

SRS-NFR-04 — Security of Credentials & Session Tokens *[Priority: Critical]*	
Requirement Statement	All API keys and secrets must be isolated from source control, and all authentication tokens must be single-use, time-boxed, and role-scoped.
Rationale	Magic-link authentication and third-party API keys are both high-value targets; leaking either would compromise candidate data or the team's OpenAI billing.
System Behavior / Implementation	• OpenAI keys and database credentials are loaded exclusively from environment variables (.env), never committed to the repository. • Magic-link tokens expire after a defined short window and are invalidated on first use. • Session credentials are role-scoped so a candidate token can never access recruiter-only routes.
Verification / Fit Criterion	A repository secret scan finds zero committed credentials; a reused or expired magic link is rejected 100% of the time in testing.

SRS-NFR-05 — Usability for Non-Technical Candidates *[Priority: High]*	
Requirement Statement	A candidate must be able to complete the entire interview flow using only a modern web browser and a microphone, with no account creation, software installation, or technical configuration.
Rationale	Candidates are external users with varying technical comfort; friction at this stage directly reduces completion rates and produces a worse dataset for recruiters.
System Behavior / Implementation	• The interview workspace uses a single-column, distraction-minimized layout built with Tailwind CSS. • Microphone permission prompts, timers, and submission states use plain language rather than technical jargon. • The full flow (consent through submission) requires no more than the initial magic-link click and standard browser permission grants.

SRS-NFR-05 — Usability for Non-Technical Candidates *[Priority: High]*	
Verification / Fit Criterion	A first-time user with no prior briefing can complete a full test interview session without external help, verified through informal usability testing with non-team volunteers.

SRS-NFR-06 — Reliability & Graceful Degradation *[Priority: High]*	
Requirement Statement	Partial failures in an individual pipeline stage (transcription, follow-up generation, or scoring) must not corrupt or block the rest of the candidate's session.
Rationale	Cloud AI dependencies can intermittently fail or return malformed output; the system must degrade gracefully rather than losing an entire candidate's interview.
System Behavior / Implementation	• Each AI call is wrapped in error handling that persists a clear failure state rather than leaving a null or half-written database row. • A failed transcription or grading step routes the session to manual recruiter review instead of silently discarding data. • Retries are attempted once for transient/schema failures before falling back to a flagged state.
Verification / Fit Criterion	In fault-injection testing (simulated API errors), 100% of affected sessions still produce a reviewable, non-corrupted record for the recruiter.

SRS-NFR-07 — Maintainability via API Contract Discipline *[Priority: Medium]*	
Requirement Statement	Frontend and backend teams must be able to build against a fixed, versioned API/data contract defined before implementation begins, without waiting on the live AI pipeline.
Rationale	With a 3-month timeline and only ~18 hours/week per team member, parallel workstreams are essential; undocumented or shifting contracts would create integration bottlenecks late in the project.
System Behavior / Implementation	• Request/response schemas for every endpoint are documented in Week 1 before feature coding starts. • The frontend team builds against mock/fixture data matching the agreed schema while backend and AI integration proceed independently. • Any schema change after Week 1 is treated as a tracked decision, not an implicit change.
Verification / Fit Criterion	Frontend and backend components integrate in Month 2 with no contract-mismatch defects traced to undocumented schema changes.

SRS-NFR-08 — Cross-Browser & Cross-Platform Portability *[Priority: Medium]*	
Requirement Statement	Core interview functionality — microphone recording, timers, and anti-cheat listeners — must function correctly on current versions of Chrome, Firefox, and Safari on both Windows and macOS.

SRS-NFR-08 — Cross-Browser & Cross-Platform Portability *[Priority: Medium]*	
Rationale	Candidates use whatever device and browser they already have; the product cannot assume a specific browser or OS.
System Behavior / Implementation	• MediaRecorder MIME type selection includes an explicit Safari-compatible fallback. • Page Visibility API and clipboard event handling are tested against each target browser's known quirks. • No functionality depends on browser extensions or non-standard APIs.
Verification / Fit Criterion	A defined smoke-test checklist passes on Chrome, Firefox, and Safari across at least one Windows and one macOS device before the final deployment milestone.

SRS-NFR-09 — Scoped Scalability for a Bounded MVP Cohort *[Priority: Low]*	
Requirement Statement	The system must reliably support the concurrency expected of a university-project pilot cohort (a small number of simultaneous candidate sessions) without requiring horizontal scaling infrastructure.
Rationale	Building for enterprise-scale concurrency within a 3-month, 4-person, $10/month-budget constraint would consume time better spent on core interview quality; scalability beyond the pilot cohort is explicitly out of scope for this version.
System Behavior / Implementation	• Deployment targets a single hosted instance (e.g., Render or Railway) sized for light concurrent load. • Database indexing is applied to the primary lookup paths (Candidate_ID, Job_ID) to keep dashboard queries responsive at pilot scale. • No load-balancing, queueing infrastructure, or multi-region deployment is attempted in this version.
Verification / Fit Criterion	The system remains responsive (no dropped recordings or failed uploads) under a test load matching the expected pilot cohort size.

6. Appendix: Development Matrices

6.1 Operational Task Assignment

The 3-month timeline is divided into four sequential phases, each spanning roughly three to four weeks. Every team member carries responsibilities in every phase, but with a consistent silo focus that lets frontend, backend, and AI/infrastructure work proceed in parallel using the Week 1 API contract described in SRS-NFR-07. The Project Lead's responsibilities shift from design (Phase 1) to prompt engineering (Phase 2) to quality assurance (Phase 3) to release management (Phase 4).

Phase	Project Lead (Thaing Parinha)	Frontend Engineer (Soeng Senghorng)	Backend Engineer (Lim Hokan)	AI/Infrastructure Engineer (Uy Sovannareach)
Phase 1 (Month 1)	Preparation & system design: sprint plan, scoring rubric design, Base Question definitions.	Setup & Candidate Workspace UI: React project setup, Tailwind layout, MediaRecorder audio capture.	Database Setup & Authentication: FastAPI workspace, PostgreSQL migrations, relational Foreign Keys.	Local Setup, Staging & API Foundations: .env key isolation, OpenAI SDK integration, Render/Railway hosting.
Phase 2 (Month 2, early)	Prompt engineering & AI setup: system prompts for dynamic follow-up and JSON grading.	System Connection & Anti-Cheat Guardrails: FastAPI linking, clipboard lock, Page Visibility API tracking.	Audio Upload & Core API Routes: consent logger, tab-out listeners, 20MB upload validator.	Speech-to-Text & Follow-Up Chain: Whisper transcription logic, async/await request wrappers.
Phase 3 (Month 2, late)	QA & Testing: prompt-injection/jailbreak testing, edge cases (empty files, wrong language), scoring consistency at temperature 0.0.	Recruiter Dashboard & Scoreboard UI: ranked leaderboard table, split-panel transcript/score drawers.	AI Hand-off & Data Aggregation: asynchronous data routes to AI pipelines without triggering browser timeouts.	JSON Grading Engine & Cost Guards: response_format enforcement for 1–5 scoring, token monitoring middleware.
Phase 4 (Month 3)	Polish & Deploy: anti-cheat UI flag testing, backup demo video recording for presentation day.	Integration Testing & Polish: Safari/Chrome cross-browser compatibility, audio element fixes.	Security Polish & Cost Tracking: API daily ceiling caps, database indexing stress tests.	Final Staging Deployment & AI Optimization: API calibration, production pipeline deployment.

6.2 Quick-Response Defensive Cheat Sheet

The following matrix maps each anticipated threat category — candidate fraud, prompt injection, and runaway token/cost loops — to the specific structural engineering control already designed into this specification, with cross-references to the relevant functional and non-functional requirements. This table is intended as a fast-reference for the Project Lead's Phase 3 QA testing pass.

Threat Vector	Structural Engineering Response
Candidate Fraud — Pasting Pre-Written Answers	Global paste (Ctrl+V/Cmd+V) and right-click context-menu interception (FR-11) forces manual typing, which consumes the tight timer windows (FR-05, FR-09) and structurally discourages copy-paste cheating.
Candidate Fraud — Searching for Answers Off-Screen	Page Visibility API tab-out tracking (FR-12) logs every blur event to the immutable AuditLogs table and surfaces a cumulative count on the recruiter dashboard (FR-14), converting a silent behavior into a visible risk signal.
Candidate Fraud — Using an External AI Chatbot for the Follow-Up	The 2:30 hardcoded follow-up window (FR-09) is deliberately shorter than the time needed to read a surprise technical question, type it into an external tool, wait for a reply, and read the answer back.
Candidate Fraud — Reciting Memorized/Templated Answers	The GPT-4o-mini grading prompt (FR-03, FR-10) explicitly instructs the model to detect rigid textbook transitions and repetitive structural patterns and down-weight the Communication trait accordingly, with the trigger recorded in the rationale.
Prompt Injection — Candidate Embeds Instructions in Spoken/Transcribed Answers	Transcripts are always passed to GPT-4o-mini as clearly delimited user-content data inside a fixed system prompt, never concatenated into the instruction layer; response_format: json_object further constrains the model to a rigid schema that resists being redirected into free-form compliance.
Prompt Injection — Model Produces Off-Schema or Manipulated Output	Every AI response is validated against the expected JSON schema (FR-03) before being persisted; a failed validation triggers one corrective retry, then falls back to a manual-review flag (FR-15) rather than trusting unvalidated output.
Prompt Injection / Consistency — Score Drift Across Runs	Grading calls are pinned to temperature 0.0 (Project Lead Phase 3 QA duty) to minimize run-to-run variance, and the Project Lead runs dedicated jailbreak/injection/consistency test batches before deployment.
Token Loop / Runaway Cost — Buggy Retry Loop	The 15-second asynchronous processing wrapper (FR-17) prevents a stalled request from being fired repeatedly by an impatient client, and background jobs are tracked by state rather than re-triggered on every client poll.
Token Loop / Runaway Cost — Unexpected Volume Spike	The Budget Kill-Switch (FR-16, NFR-03) tracks running token/cost estimates in backend middleware and freezes all outbound AI calls once the $10.00 monthly ceiling is reached, backed by an independent hard cap set directly in the OpenAI platform dashboard.
Data Quality — Empty, Silent, or Foreign-Language Audio	The Whisper transcription step (FR-07) explicitly flags empty/no-speech results instead of fabricating a transcript, so downstream grading never scores a candidate against invented text.
AI Grading Error — Hallucinated or Unfairly Harsh Score	The Human-in-the-Loop override (FR-15) flags low-communication or high-tab-out sessions for manual review,

Threat Vector	Structural Engineering Response
	and every session's raw audio remains directly playable from the recruiter dashboard for a final human judgment.

End of Document.